

# The Transformer Architecture

A Complete Step-by-Step Mathematical Walkthrough from Scratch

Comprehensive Beginner-to-Advanced Guide

# Contents

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| <b>1</b> | <b>Data Preparation &amp; Tokenization</b>                 | <b>1</b>  |
| 1.1      | The Vocabulary . . . . .                                   | 1         |
| 1.2      | The Tokenization Functions . . . . .                       | 1         |
| <b>2</b> | <b>The Embedding Space &amp; Positional Encoding</b>       | <b>3</b>  |
| 2.1      | Token Embeddings . . . . .                                 | 3         |
| 2.2      | Positional Encoding (PE) . . . . .                         | 3         |
| 2.2.1    | Combining and Dropout . . . . .                            | 3         |
| <b>3</b> | <b>Multi-Head Self-Attention</b>                           | <b>5</b>  |
| 3.1      | Creating Queries, Keys, and Values . . . . .               | 5         |
| 3.2      | Splitting into Heads . . . . .                             | 5         |
| 3.3      | The Attention Calculation . . . . .                        | 5         |
| 3.3.1    | 1. Dot Product (Similarity) . . . . .                      | 5         |
| 3.3.2    | 2. Scaling . . . . .                                       | 6         |
| 3.3.3    | 3. Masking . . . . .                                       | 6         |
| 3.3.4    | 4. Softmax . . . . .                                       | 6         |
| 3.3.5    | 5. Weighting Values and Merging . . . . .                  | 6         |
| <b>4</b> | <b>The Encoder Stack</b>                                   | <b>7</b>  |
| 4.1      | Add & Norm (Residuals and Layer Normalization) . . . . .   | 7         |
| 4.2      | Feed Forward Network (FFN) . . . . .                       | 7         |
| <b>5</b> | <b>The Decoder Architecture</b>                            | <b>9</b>  |
| 5.1      | Masked Self-Attention (The Causal Mask) . . . . .          | 9         |
| 5.2      | Encoder-Decoder Cross-Attention . . . . .                  | 9         |
| 5.3      | Final FFN . . . . .                                        | 9         |
| <b>6</b> | <b>The Full Transformer Assembly &amp; Backpropagation</b> | <b>11</b> |
| 6.1      | The Final Projection Layer . . . . .                       | 11        |
| 6.2      | Intuition on Backpropagation . . . . .                     | 11        |

# Chapter 1

## Data Preparation & Tokenization

Before a neural network can process text, we must convert raw strings into numerical formats. This chapter covers the exact tokenization pipeline used in our PyTorch notebook.

### 1.1 The Vocabulary

The notebook defines a vocabulary of 39 distinct tokens. Special tokens dictate sequence structure:

- `<pad>` (ID 0): Padding token, used to ensure uniform sequence lengths.
- `<unk>` (ID 1): Unknown token, for words outside the vocabulary.
- `<cls>` (ID 2): Start-of-Sequence / Classification token.
- `[sep]` (ID 3): Separator token.

### 1.2 The Tokenization Functions

The codebase breaks tokenization into modular steps. Let's trace the input: `"anugrah is learning"`. We set `max_seq_len = 10`.

#### Code Breakdown: Function 1: `_add_special_tokens_and_segments`

**Input:** `"anugrah is learning"`

**Logic:** Splits the string by spaces, appends `<cls>` at the start and `[sep]` at the end. All tokens get assigned a segment ID of 0.

**Output Tokens:** `['<cls>', 'anugrah', 'is', 'learning', '[sep]']`

#### Code Breakdown: Function 2: `_convert_to_ids`

**Input:** `['<cls>', 'anugrah', 'is', 'learning', '[sep]']`

**Logic:** Looks up each string in the `new_vocab` dictionary.

**Edge Case in Notebook:** The code uses `vocab.get("<cls>", "<cls>")` which defaults to the string `"<cls>"` rather than ID 2. When `_convert_to_ids` runs, it doesn't find the string `"<cls>"` natively handled, defaulting to `<unk>` (ID 1).

**Output IDs:** `[1, 38, 5, 9, 1]`

#### Code Breakdown: Function 3: `_pad_ids_and_segments`

**Logic:** The sequence length is 5. `max_seq_len` is 10. We pad 5 zeros to the end.

**Output Padded IDs:** `[1, 38, 5, 9, 1, 0, 0, 0, 0, 0]`

**Code Breakdown: Function 4: `_create_attention_mask_from_ids`**

**Logic:** Creates a binary map where 1 = Real Token, 0 = Padding. Then it unsqueezes dimensions to match the expected multi-head attention shape `[batch_size, 1, 1, seq_len]`.

**Output Mask:** `[[[1, 1, 1, 1, 1, 0, 0, 0, 0]]]`

**Vector Tracking: Outputs of Data Prep**

At the end of `prepare_transformer_inputs`, our tensors are ready for the network:

- **Input IDs:** Shape `[1, 10]`. Values: `[1, 38, 5, 9, 1, 0, 0, 0, 0, 0]`
- **Token Type IDs:** Shape `[1, 10]`. Values: `[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]`
- **Attention Mask:** Shape `[1, 1, 1, 10]`.

**Interview Questions: Chapter 1**

**Q1: Why do we need the `<pad>` token, and why must it be masked later?**

A: Neural networks process data in batches of fixed-size tensors. Shorter sentences must be padded to match the longest sentence in the batch. However, padding tokens contain no semantic meaning. If we don't mask them, the Attention mechanism will "pay attention" to these zeros, corrupting the contextual meaning of real words.

**Q2: What is the purpose of `token_type_ids` (Segment IDs)?**

A: In models like BERT, tasks like Next Sentence Prediction require processing two sentences simultaneously. Segment IDs (0 for Sentence A, 1 for Sentence B) tell the model which token belongs to which sentence.

## Chapter 2

# The Embedding Space & Positional Encoding

Words are now integers, but integers lack semantic distance. We must project them into an  $n$ -dimensional space where geometric distances represent semantic similarity.

### 2.1 Token Embeddings

We instantiate `nn.Embedding(num_embeddings=39, embedding_dim=8)`. This initializes a  $39 \times 8$  weight matrix. Passing our `[1, 10]` sequence through this layer results in a tensor of shape `[1, 10, 8]`.

Example lookup for ID 38 ("anugrah"):

$$E_{word} = [0.1271, -1.5969, 1.5901, 0.3746, 0.9116, -0.6469, -0.4991, 0.5582]$$

### 2.2 Positional Encoding (PE)

Self-Attention processes all tokens simultaneously. It has no built-in notion of sequence order. We inject order using sine and cosine waves of varying frequencies.

#### Mathematical Insight: Positional Encoding Math

For a given position  $pos$  and dimension index  $i \in [0, d_{model}/2)$ :

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$
$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Let's calculate  $PE$  for the word "anugrah", which is at position  $pos = 1$ , for dimensions  $i = 0$  and  $i = 1$  ( $d_{model} = 8$ ):

- Dim 0 ( $2i = 0$ ):  $\sin(1/10000^0) = \sin(1) = 0.8415$
- Dim 1 ( $2i + 1 = 1$ ):  $\cos(1/10000^0) = \cos(1) = 0.5403$
- Dim 2 ( $2i = 2$ ):  $\sin(1/10000^{2/8}) = \sin(1/10) = 0.0998$

#### 2.2.1 Combining and Dropout

The Positional Encoding is directly added to the word embedding:  $X = E_{word} + PE$ . In the notebook, `self.dropout` ( $p = 0.1$ ) is applied. To keep the expected value constant during training, PyTorch mathematically scales non-dropped elements by  $\frac{1}{1-0.1} = 1.1111$ . This explains the numerical shift in the notebook's final tensor outputs. Finally, Segment Embeddings are added.

**Vector Tracking: Output of Embedding Space**

**Input Tensor:** [1, 10]

**Output Tensor:** [1, 10, 8]

This tensor represents the rich, order-aware representation of our text, ready to enter the Encoder.

**Interview Questions: Chapter 2**

**Q1: Why use alternating sines and cosines instead of just adding integer indices (1, 2, 3...)?**

A: Integer indices grow unboundedly, distorting the vector magnitudes for long sequences. Sine/cosine waves ensure values remain bounded between  $[-1, 1]$ . Additionally, their periodic nature allows the model to easily learn relative positions (token  $A$  is  $k$  steps away from token  $B$ ).

**Q2: Why do we add Positional Encodings to Word Embeddings instead of concatenating them?**

A: Concatenation would increase the dimensionality (e.g., from 8 to 16), increasing computational cost. Adding them operates on the principle that high-dimensional vectors are mostly orthogonal; the model can linearly separate the "what" (word meaning) from the "where" (position) without information destruction.

# Chapter 3

## Multi-Head Self-Attention

This is the heart of the Transformer. Self-Attention allows each word to look at every other word in the sequence to gather context.

### 3.1 Creating Queries, Keys, and Values

The input  $X$  (shape  $[1, 10, 8]$ ) is multiplied by three learnable weight matrices:  $W_Q, W_K, W_V \in \mathbb{R}^{8 \times 8}$ .

$$Q = XW_Q \implies \text{Shape: } [1, 10, 8]$$

$$K = XW_K \implies \text{Shape: } [1, 10, 8]$$

$$V = XW_V \implies \text{Shape: } [1, 10, 8]$$

*Intuition:* **Query:** What am I looking for? **Key:** What do I contain? **Value:** What information will I pass on?

### 3.2 Splitting into Heads

To allow the model to focus on different linguistic aspects (e.g., grammar vs. vocabulary) simultaneously, we split the 8 dimensions into `num_heads=2`. Each head gets `head_dim = 4`.

#### Vector Tracking: Reshaping for Heads

**Before:**  $[1, 10, 8]$

**Reshape & Transpose:**  $[1, 2, 10, 4]$

(Batch = 1, Heads = 2, SeqLen = 10, Dim = 4)

### 3.3 The Attention Calculation

#### Mathematical Insight: Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V$$

#### 3.3.1 1. Dot Product (Similarity)

We compute  $Q \times K^T$ . Shape math:  $(10 \times 4) \times (4 \times 10) \implies 10 \times 10$ . This  $10 \times 10$  matrix represents raw "scores" of how much every word aligns with every other word.

### 3.3.2 2. Scaling

We divide the scores by  $\sqrt{d_k} = \sqrt{4} = 2$ . *Why?* If dot products grow too large, the softmax function will be pushed into regions with extremely small gradients (vanishing gradients), halting learning.

### 3.3.3 3. Masking

We apply the padding mask `attention_mask_new` ([[[[1, 1, 1, 1, 1, 0, 0, 0, 0, 0]]]]). Where the mask is 0, we overwrite the score with  $-\infty$ .

### 3.3.4 4. Softmax

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum e^{x_j}}$$

Because  $e^{-\infty} = 0$ , the padding tokens get exactly 0 attention. The rows of the  $10 \times 10$  matrix now sum to 1.0.

### 3.3.5 5. Weighting Values and Merging

We multiply the  $10 \times 10$  attention weights by the  $V$  matrix ( $10 \times 4$ ) to yield a ( $10 \times 4$ ) output per head. We transpose and concatenate the heads back together:  $[1, 2, 10, 4] \Rightarrow [1, 10, 8]$ . Finally, we pass it through a linear projection  $W_O$ .

#### Interview Questions: Chapter 3

##### Q1: What are Q, K, and V analogous to in real life?

A: Think of a library. The **Query** is your search term ("machine learning books"). The **Keys** are the book titles/tags on the shelves. The **Values** are the actual contents of the books. You compute the similarity between your Query and all Keys, and extract the Values from the most similar ones.

##### Q2: Why do we use Multi-Head Attention instead of Single-Head Attention with higher dimensions?

A: Single-head attention averages out all context. By splitting into multiple heads, Head 1 might learn to attend to the grammatical subject of the sentence, while Head 2 learns to attend to emotional sentiment, capturing richer representations simultaneously without extra parameter cost.



# Chapter 4

## The Encoder Stack

The Self-Attention mechanism is wrapped in additional layers to stabilize training and extract deeper features. This constitutes one complete `TransformerEncoderLayer`.

### 4.1 Add & Norm (Residuals and Layer Normalization)

After Multi-Head Attention (MHA), we use a Residual Connection followed by Layer Normalization:

$$x_{new} = \text{LayerNorm}(x_{original} + \text{MHA}(x_{original}))$$

#### Mathematical Insight: Layer Normalization

$$y = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \gamma + \beta$$

Unlike Batch Normalization which normalizes across the batch dimension, Layer-Norm normalizes across the feature dimension (our 8 dimensions) for each token individually.  $\gamma$  and  $\beta$  are learnable parameters.

### 4.2 Feed Forward Network (FFN)

The normalized output passes through a position-wise Feed Forward Network.

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$

In our code:

- $W_1$  expands the dimension: `Linear(8, 32)`
- ReLU activation introduces non-linearity.
- $W_2$  contracts the dimension: `Linear(32, 8)`

This expansion-contraction phase acts as a key-value memory bank, mapping the contextualized features into higher-level representations.

This is followed by a second **Add & Norm** step. The final shape exiting the Encoder is still `[1, 10, 8]`.

#### Interview Questions: Chapter 4

**Q1: Why are Residual Connections (Add) essential in deep Transformers?**

A: In deep networks, gradients multiply during backpropagation. If they are less than 1, they vanish rapidly, meaning early layers never learn. The Add operation ( $x + f(x)$ ) allows gradients to bypass the complex math and flow directly backward unaltered, solving the Vanishing Gradient Problem.

**Q2: Why do we need the Feed Forward Network if Attention already captures context?**

A: Self-Attention merely mixes existing vectors together linearly based on context. The FFN applies non-linear transformations (ReLU) independently to each token, processing the \*result\* of the attention mixing. If Attention gathers the ingredients, FFN bakes the cake.

## Chapter 5

# The Decoder Architecture

The Decoder generates the output sequence autoregressively (one word at a time). Our `TransformerDecoderLayer` has three sub-layers.

### 5.1 Masked Self-Attention (The Causal Mask)

During training, we feed the entire target sequence into the decoder. However, the model must not "cheat" by looking ahead at words it hasn't predicted yet. We enforce this using a **Causal Mask**.

$$\text{Causal Mask} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

When this mask is applied to the  $(10 \times 10)$  attention scores, all upper-triangular zeros become  $-\infty$ . After softmax, token at position  $t$  can only attend to tokens  $\leq t$ .

### 5.2 Encoder-Decoder Cross-Attention

This layer connects the Decoder to the Encoder.

- **Query ( $Q$ ):** Comes from the output of the Decoder's previous sub-layer. (*What should the output sequence look for next?*)
- **Key ( $K$ ) and Value ( $V$ ):** Come from the **Encoder's Final Output**. (*Here is the contextualized input sequence.*)

#### Vector Tracking: Cross Attention Shapes

If our Encoder output is `[Batch, SrcLen, Dim]` and Decoder input is `[Batch, TgtLen, Dim]`:

- $Q$  Shape: `[1, 2, TgtLen, 4]`
- $K^T$  Shape: `[1, 2, 4, SrcLen]`
- Dot Product  $\implies$  `[1, 2, TgtLen, SrcLen]`

This matrix perfectly maps every target token to the most relevant input source tokens!

### 5.3 Final FFN

Just like the Encoder, the Decoder ends with an FFN and Add & Norm. The output shape is `[1, 10, 8]`.

**Interview Questions: Chapter 5****Q1: Contrast the Padding Mask and the Causal Mask.**

A: The Padding Mask is used in the Encoder to hide '<pad>' tokens from attention. The Causal Mask is used in the Decoder to hide "future" tokens from the current token during training, preserving the autoregressive property. Cross-attention uses the padding mask from the source sequence.

**Q2: Where do Q, K, and V come from in Cross-Attention?**

A: Q comes from the Decoder (representing the current translation state). K and V come from the Encoder (representing the source text memory). This forces the decoder to extract relevant parts of the input text based on what it is currently trying to generate.

## Chapter 6

# The Full Transformer Assembly & Backpropagation

We assemble the `FullTransformer` module by combining the Embedding layers, the Encoder stack, the Decoder stack, and a final Linear projection.

### 6.1 The Final Projection Layer

The Decoder's final output tensor has shape  $[1, 10, 8]$ . However, our goal is to predict the next word from a vocabulary of 39 words. We use a **Linear Layer**:

$$\text{Logits} = X_{\text{decoder\_out}} W_{\text{out}}$$

Where  $W_{\text{out}} \in \mathbb{R}^{8 \times 39}$ .

#### Vector Tracking: Final Model Output

$$[1, 10, 8] \times [8, 39] \implies [1, 10, 39]$$

For each of the 10 token positions, we now have 39 raw scores (logits). Applying Softmax to these 39 values gives a probability distribution over the vocabulary. The word with the highest probability is the model's prediction.

### 6.2 Intuition on Backpropagation

How does the network learn to update  $W_Q$ ,  $W_K$ , and  $W_V$ ?

1. **Loss Calculation:** We compare the output probabilities against the actual target words using Cross-Entropy Loss.
2. **Gradient Flow:** The error gradient flows backwards through the Final Linear layer into the Decoder.
3. **Through Cross-Attention:** In Cross-Attention, the gradient splits. One path flows back into the Decoder to update the Decoder's queries ( $W_Q$ ). The other path flows all the way back into the **Encoder** to update how the Keys and Values ( $W_K, W_V$ ) are generated.
4. **Through Residuals:** The Add layers act as "gradient super-highways," ensuring the error signal remains strong all the way back to the initial Embedding layers.

#### Interview Questions: Chapter 6

**Q1: Why is Softmax applied at the very end?**

A: Softmax converts the raw unconstrained outputs (logits) of the Linear layer into

a normalized probability distribution (values between 0 and 1 that sum to 1). This matches the format expected by the Cross-Entropy loss function, which penalizes the model based on how far its predicted probabilities deviate from the true labels.

**Q2: Can we train a Transformer without Positional Encoding?**

A: It will run and compile, but its performance will be catastrophic. Because Multi-Head Attention processes text entirely as an unordered "bag of words", the sentences "The dog chased the cat" and "The cat chased the dog" would produce the exact same mathematical representations without positional encoding.